

Recursive Reward Aggregation の CVaR 最適化への応用

演習Ⅲ第1期 杉山・石田研究室

東海林 明

May 15, 2026

演習 3 でやったこと

強化学習の基礎の学習と金融分野での応用について調べ、既存の論文実装にプラスアルファを実装した。

- まずは強化学習について、「ゼロから作る Deep Learning 4 強化学習編」で勉強
- 次に、論文「Recursive Reward Aggregation」を読んで内容を理解
- 論文の実装をベースに CVaR を最適化する実装を行った

目次

- ・ イントロ
 - ・ 強化学習
 - ・ ポートフォリオ最適化
- ・ 題材とした研究
- ・ やったこと
- ・ 最後に

- **強化学習 (Reinforcement Learning)** : エージェントが環境と相互作用しながら、報酬を最大化する最適な行動を学習
- **主な要素** :
 - エージェント : 意思決定を行う主体
 - 環境 : エージェントが相互作用する世界
 - 状態 S : 環境の状態
 - 行動 A : エージェントが実行可能な選択肢
 - 報酬 R : 行動の結果に対するフィードバック
- **目標** : 期待される累計報酬を最大化する方策 π を学習

マルコフ決定過程 (MDP)

マルコフ決定過程 (MDP) は、以下の要素から構成される。

- S : 状態空間 (有限または連続)
- A : 行動空間 (有限または連続)
- $p(s'|s, a)$: 遷移確率 状態 s で行動 a を取ったとき、次の状態 s' に遷移する確率 (決定的なら $s' = p(s, a)$)
- $r(s, a)$: 報酬関数
- $\pi(a|s)$: 方策 (決定的なら $a = \pi(s)$)

なお、状態は現在の状態と行動のみに依存し、過去の履歴には依存しない (マルコフ性)。

ベルマン方程式

累計報酬として、割引率 $\gamma \in [0, 1]$ を用いた割引報酬和を考える場合の状態価値関数 $v^\pi(s)$ は、以下のように定義される。

$$v^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1}) \mid s_0 = s \right]$$

ベルマン方程式：

$$v^\pi(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma v^\pi(s')]$$

⇒ 再帰的構造を活かして、ある方策の下での状態価値を連立方程式から計算可能¹

¹実際には連立方程式を使った方法は実践的には計算量が膨大になってしまい使えず、あくまでベルマン方程式は強化学習の理論における数学的な基礎を与えるものである。

最適ベルマン方程式

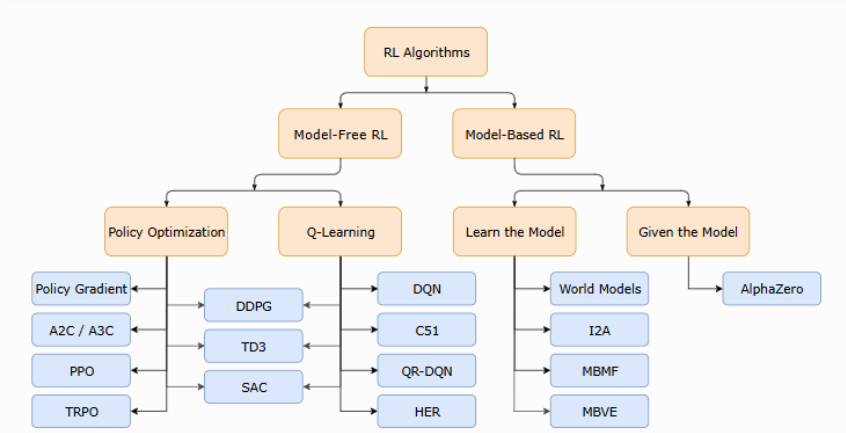
最適方策 π^* : 全ての状態で最大の期待累計報酬を得る方策

最適方策の下での状態価値 $v^*(s)$ は、以下の最適ベルマン方程式を満たす。²

$$v^*(s) = \max_a \sum_{s'} p(s'|s, a) [r(s, a, s') + \gamma v^*(s')]$$

²ここまでのベルマン方程式と最適ベルマン方程式には、行動価値関数 $q^\pi(s, a)$ を用いたバージョンも存在する。

強化学習 (深層) の手法の分類



画像：OpenAI 2026

主に**方策勾配法**が金融分野でのポートフォリオ最適化などのタスクに適用されることが多い。

理由:

- 金融タスクでは市場の環境モデルを正確に構築するのが難しい
- 金融タスクの多くは連続的な行動空間が必要

方策勾配法の概要

アイデア：価値関数ではなく、方策 π を直接学習する

パラメータ化された方策：パラメータ θ でパラメータ化された方策 $\pi_\theta(a|s)$ 、通常、ニューラルネットワークなどで表現される

目的関数： $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$ を軌跡としたとき、累計報酬は $G(\tau)$ で計算できるとすれば、

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)]$$

学習：目的関数 $J(\theta)$ を最大化するようにパラメータ θ を勾配法で更新

目的関数 $J(\theta)$ の勾配は以下のように導出される。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

これは、 τ をサンプルし、実際に得られたデータを平均することで推定することができる (モンテカルロ法)。

基本の方策勾配法から、勾配を、

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

と $G(\tau)$ の代わりに G_t すなわち、時刻 t 以降の累計報酬を用いるようにしたものが **REINFORCE** アルゴリズムである。

特徴：

- 行動の前の報酬が行動の勾配に影響を与えないため、より正確
- 分散が大きく学習が安定しない

ベースラインを用いた分散削減

報酬から状態に依存して決まる基準となる値（ベースライン）を差し引いても勾配が変わらない性質を利用する。

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T (G_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

特徴：

- イメージとしては状態に対してその状態から期待される報酬みたいなものの差し引きで行動を評価する
- ベースラインを適切に選ぶことで、勾配の分散を大幅に削減できる

さらに、別のニューラルネットなどで TD 学習 (すなわち、 $v_\phi(s_t)$ が $r_t + \gamma v_\phi(s_{t+1})$ に近づくような学習) がされた価値関数 $v_\phi(s)$ をベースラインとして使用し、さらに $G_t = r_t + \gamma v_\phi(s_{t+1})$ として、

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T (r_t + \gamma v_\phi(s_{t+1}) - v_\phi(s_t)) \nabla_\theta \log \pi_\theta(a_t | s_t) \right]$$

とすれば、エピソード全体の報酬を待たずに 1 ステップ先の結果さえ見ればその場で方策を更新することができる。

特徴：

- 1 ステップ先しか見ないので分散が大きくなり学習が安定する
- 一方で、 $v_\phi(s)$ が正しく学習されないことによるバイアスが生じる

目次

- ・ イントロ
 - ・ 強化学習
 - ・ ポートフォリオ最適化
- ・ 題材とした研究
- ・ やったこと
- ・ 最後に

ポートフォリオ最適化タスクの概要

ポートフォリオ最適化：複数の資産に対する投資配分を最適化する問題

問題設定：

- n 種類の資産（株式、債券など）
- 配分ウェイト $\mathbf{w} = (w_1, w_2, \dots, w_n)$, $\sum_i w_i = 1$
- 各時刻 t でのリターン $\mathbf{r}_t = (r_{1,t}, r_{2,t}, \dots, r_{n,t})$
- ポートフォリオリターン： $R_t = \sum_i w_i r_{i,t}$

RL での定式化：

- 状態：過去のリターンや市場情報
- 行動：資産配分ウェイト
- 報酬：ポートフォリオのリターンまたは目的関数

Sharpe Ratio

R をリターン系列としたとき、**Sharpe Ratio** は以下のように定義される。

$$\text{Sharpe Ratio} = \frac{\mathbb{E}[R]}{\sigma[R]}$$

ここで $\mathbb{E}[R]$ は期待リターン、 $\sigma[R]$ は標準偏差

特徴：

- リスク (標準偏差) は小さく、リターンは大きいほうが良い
- 高いほどいいポートフォリオとなる
- 累積的に計算することはできない

Sharpe Ratio は累積的に計算できないため、目的とするには以下のような手法が用いられる。

従来手法と課題：

- **DiffSharpe**：微分可能な差分 Sharpe ratio を報酬として使用→ 学習目的と実際の Sharpe ratio に矛盾が生じる
- **NCMDP**：履歴を状態に含める方法→ 状態空間が爆発的に増加、計算効率が低下

目次

- ・ イントロ
- ・ 題材とした研究
- ・ やったこと
- ・ 最後に

主要なアイデア：報酬関数そのものを修正するのではなく、得られた報酬の「集約方法 (Aggregation Function)」を変更することで、複雑な目的を最適化する。

特徴：

- MDP を代数的な視点から再定式化
- 報酬の生成と集約から、ベルマン方程式が自然に導出される枠組みを構築
- 従来の「割引報酬和」という線形な集約演算を、より一般的な非線形な再帰的集約へと拡張
- 目的関数が再帰的構造を持てば、状態空間を拡張することなく動的計画法や深層強化学習 (Actor-Critic 等) にシームレスに統合可能

通常が決定的 MDP :

$$s_{t+1} = p(s_t, a_t)$$

$$a_t = \pi(s_t)$$

$$r_t = r(s_t, a_t)$$

関数合成による表記：行動を経由せずに書き直す

$$p_{\pi}(s) := p(s, \pi(s))$$

$$r_{\pi}(s) := r(s, \pi(s))$$

これにより状態のみに依存する関数として統一的に表現できる。

報酬列生成関数 (gen) の再帰的定義

状態 s から報酬列を生成する関数 gen を再帰的に定義する。終端状態を S_ω 、リスト操作を cons で表す。

$$\text{gen}_{\pi,r,\omega}(s) := \begin{cases} [] & \text{if } s = S_\omega \\ \text{cons}(r_\pi(s), \text{gen}_{\pi,r,\omega}(p_\pi(s))) & \text{otherwise} \end{cases}$$

この関数は初期状態 s からエピソード終了まで軌跡を辿り、報酬列 $[r_0, r_1, \dots, r_T]$ を生成する。

通常の RL でのベルマン方程式導出

再帰的に定義される関数 sum_γ (割引報酬和) を以下のように定義する。

$$\text{sum}_\gamma([]) := 0$$

$$\text{sum}_\gamma([r_0, r_1, \dots]) := r_0 + \gamma \cdot \text{sum}_\gamma([r_1, r_2, \dots])$$

価値関数は 2 つの再帰関数の合成として表現できる。

$$v_\pi(s) = \text{sum}_\gamma(\text{gen}_{\pi, r, \omega}(s))$$

sum_γ が再帰的であるため、

$$v_\pi(s) = \begin{cases} 0 & \text{if } s = S_\omega \\ r_\pi(s) + \gamma v_\pi(p_\pi(s)) & \text{otherwise} \end{cases}$$

これがベルマン方程式の導出。

観察：

ベルマン方程式が成り立つ理由は sum_γ が再帰的だから。

一般化：

sum_γ でなくても、任意の再帰的関数であれば同じ構造の方程式が成り立つのではないか？

⇒ これが Recursive Reward Aggregation (RRA) の基本的なアイデア

記号の定義：

- T : statistics (報酬を集約するための中間的表現)
- $\text{init} \in T$: 初期 statistics
- $(\triangleright) : \mathbb{R} \times T \rightarrow T$: 新しい報酬で statistics を更新する二項演算
- $\text{post} : T \rightarrow \mathbb{R}$: statistics から最終値を取り出す関数

これらを用いて、報酬列 $[r_0, r_1, \dots, r_T]$ を集約する関数 $\text{agg}_{\text{init}, \triangleright} : [\mathbb{R}] \rightarrow T$ を定義する。

$$\text{agg}_{\text{init}, \triangleright}([r_0, r_1, \dots, r_T]) = r_0 \triangleright (r_1 \triangleright (\dots \triangleright (r_T \triangleright \text{init}) \dots))$$

報酬列 $[r_0, \dots, r_T] = \text{gen}_{\pi, r, \omega}(s)$ が得られれば、目的関数の値は

$$\text{post}(\text{agg}_{\text{init}, \triangleright}(\text{gen}_{\pi, r, \omega}(s)))$$

として計算できる。

RRA の一般化ベルマン方程式

$\text{agg}_{\text{init}, \triangleright} \circ \text{gen}_{\pi, r, \omega}$ の合成に対して、 fusion^3 を適用するとこの合成は以下の再帰的関数 $\tau_\pi : \mathcal{S} \rightarrow T$ と等価になる。

$$\tau_\pi(s) := \begin{cases} \text{init} & \text{if } s = S_\omega \\ r_\pi(s) \triangleright \tau_\pi(p_\pi(s)) & \text{otherwise} \end{cases}$$

これが一般化されたベルマン方程式である。⁴

なお、価値関数は以下のようにあらわせる。

$$v_\pi(s) = \text{post}(\tau_\pi(s))$$

³ fusion とは、リストを生成する関数とリストを畳み込む関数の合成を、直接の再帰関数へと変換する代数的な変換規則。

⁴詳細は省くが、最適ベルマン方程式での導出も、確率的な MDP でも一般化可能。

RRA の枠組みでの具体例

例 1 (割引報酬和) :

- $T = \mathbb{R}$
- $\text{init} = 0$
- $r \triangleright \tau = r + \gamma \tau$
- $\text{post}(\tau) = \tau$

例 2 (Sharpe Ratio) :

- $T = (\text{count}, \text{sum}, \text{sum_sq})$
- $\text{init} = (0, 0, 0)$
- $r \triangleright (n, s, ss) = (n + 1, s + r, ss + r^2)$
- $\text{post}(n, s, ss) = \frac{s/n}{\sqrt{ss/n - (s/n)^2}}$

Aggregation Functions のバリエーション

Table 1: Recursive aggregation functions

	definition post $\circ$ agg _{init, $\triangleright$} : $[R] \rightarrow R$	initial value of statistic(s) init $\in T$	update function $\triangleright : R \times T \rightarrow T$	post-processing post : $T \rightarrow R$
discounted sum	$r_1 + \gamma r_2 + \dots + \gamma^{t-1} r_t$	discounted sum $s: 0 \in \mathbb{R}$	$+\gamma := [r, s \mapsto r + \gamma \cdot s]$	$\text{id}_{\mathbb{R}}$
discounted min	$\min\{r_1, \gamma r_2, \dots, \gamma^{t-1} r_t\}$	discounted min $n: \infty \in \overline{\mathbb{R}}$	$\min_\gamma := [r, n \mapsto \min(r, \gamma \cdot n)]$	$\text{id}_{\overline{\mathbb{R}}}$
discounted max	$\max\{r_1, \gamma r_2, \dots, \gamma^{t-1} r_t\}$	discounted max $m: -\infty \in \overline{\mathbb{R}}$	$\max_\gamma := [r, m \mapsto \max(r, \gamma \cdot m)]$	$\text{id}_{\overline{\mathbb{R}}}$
log-sum-exp	$\log(e^{r_1} + e^{r_2} + \dots + e^{r_t})$	log-sum-exp $m: -\infty \in \overline{\mathbb{R}}$	$[r, m \mapsto \log(e^r + e^m)]$	$\text{id}_{\overline{\mathbb{R}}}$
range	$\max(r_{1:t}) - \min(r_{1:t})$	max $m \begin{bmatrix} -\infty \\ \infty \end{bmatrix} \in \overline{\mathbb{R}}^2$ min n	$\left[r, \begin{bmatrix} m \\ n \end{bmatrix} \mapsto \begin{bmatrix} \max(r, m) \\ \min(r, n) \end{bmatrix} \right]$	$\left[\begin{bmatrix} m \\ n \end{bmatrix} \mapsto m - n \right]$
mean	$\bar{r} := \frac{1}{t} \sum_{i=1}^t r_i$	length $n \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in \begin{bmatrix} \mathbb{N} \\ \mathbb{R} \end{bmatrix}$ sum s	$\left[r, \begin{bmatrix} n \\ s \end{bmatrix} \mapsto \begin{bmatrix} n+1 \\ s+r \end{bmatrix} \right]$	$\left[\begin{bmatrix} n \\ s \end{bmatrix} \mapsto \frac{s}{n} \right]$
		length $n \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in \begin{bmatrix} \mathbb{N} \\ \mathbb{R} \end{bmatrix}$ mean m	$\left[r, \begin{bmatrix} n \\ m \end{bmatrix} \mapsto \begin{bmatrix} n+1 \\ \frac{n \cdot m + r}{n+1} \end{bmatrix} \right]$	$\left[\begin{bmatrix} n \\ m \end{bmatrix} \mapsto m \right]$
variance	$\frac{1}{t} \sum_{i=1}^t (r_i - \bar{r})^2 = \overline{r^2} - \bar{r}^2$	length $n \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in \begin{bmatrix} \mathbb{N} \\ \mathbb{R} \end{bmatrix}$ sum s sum square $q \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in \begin{bmatrix} \mathbb{N} \\ \mathbb{R}_{\geq 0} \end{bmatrix}$	$\left[r, \begin{bmatrix} n \\ s \\ q \end{bmatrix} \mapsto \begin{bmatrix} n+1 \\ s+r \\ q+r^2 \end{bmatrix} \right]$	$\left[\begin{bmatrix} n \\ s \\ q \end{bmatrix} \mapsto \frac{q}{n} - \left(\frac{s}{n}\right)^2 \right]$
		length $n \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \in \begin{bmatrix} \mathbb{N} \\ \mathbb{R} \\ \mathbb{R}_{\geq 0} \end{bmatrix}$ mean m variance v	$\left[r, \begin{bmatrix} n \\ m \\ v \end{bmatrix} \mapsto \begin{bmatrix} n+1 \\ \frac{n \cdot m + r}{n+1} \\ v + \frac{n(r-m)^2 - (n+1)v}{(n+1)^2} \end{bmatrix} \right]$	$\left[\begin{bmatrix} n \\ m \\ v \end{bmatrix} \mapsto v \right]$
top- k	k -th largest in $r_{1:t}$	top-1 top-2 buffer $\vdots$ $\begin{bmatrix} -\infty \\ -\infty \\ \vdots \end{bmatrix} \in \mathbb{R}^k$	$\left[r, b \mapsto \begin{cases} \text{insert}(r, b) & r > \min b \\ b & r \leq \min b \end{cases} \right]$	$[b \mapsto \min b]$

画像：Recursive Reward Aggregation(Tang et al. 2025)

実験対象：

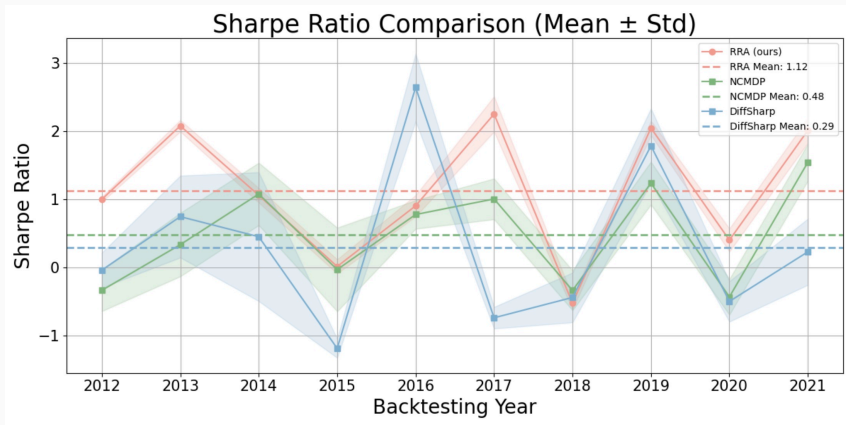
- 取引対象：12 個の S&P 500 セクターインデックス
- 追加で VIX やボラティリティに加工した特徴量も観測
- 期間：2000/1/3 2021/12/31
- 評価指標：Sharpe Ratio

学習アルゴリズム：

- ベース：PPO (Actor-Critic の一種)
- RRA の枠組みを適用するように修正

実験結果

RRA が既存手法に対して平均的に良い性能で分散も小さいことが分かった。



画像：Recursive Reward Aggregation(Tang et al. 2025)

目次

- ・ イントロ
- ・ 題材とした研究
- ・ やったこと
 - ・ 概要
 - ・ 手法
 - ・ 実験
 - ・ 結果
 - ・ 考察
- ・ 最後に

RRA の適用が可能であるためには、目的関数は再帰的構造を持って計算できる必要がある。

RRA が対応不可の目的関数：非再帰的・大域的な構造を持つもの

- **Median (中央値)：**軌跡全体を見てはじめて定義される
- **Quantile：**下位 $\alpha\%$ の値 $\rightarrow$ 軌跡全体の分布が必要

CVaR (Conditional Value at Risk) の詳細

α での CVaR は以下のように定義される。

$$\text{CVaR}_\alpha = \mathbb{E}[R \mid R \leq \text{VaR}_\alpha]$$

ここで VaR_α は下位 $\alpha\%$ に相当する損失。

特徴：

- 極端なリスクを強く考慮する
- ポートフォリオ最適化において Sharpe ほどではないが重要な指標
- 軌跡全体の報酬分布が必要なので、RRA は適用できない

CVaR を近似により RRA の枠組みで最適化する

課題：CVaR は完全には再帰的でないため、RRA の標準的な枠組みに直接適用できない

アプローチ：CVaR を再帰的に計算可能な形に近似する

- CVaR の定義： $\text{CVaR}_\alpha = \mathbb{E}[R \mid R \leq \text{VaR}_\alpha]$
- VaR_α は軌跡全体の分布に依存 $\rightarrow$ 非再帰的
- しかし、報酬の統計を **binning** といって、ヒストグラムで近似することで、再帰的に処理可能にできる。

利点：

- RRA 枠組みの汎用性を保ちながら CVaR に対応
- 状態空間の爆発を避けることができる
- 近似精度と RRA 効率のバランスを bin 数で調整可能

目次

- ・ イントロ
- ・ 題材とした研究
- ・ やったこと
 - ・ 概要
 - ・ 手法
 - ・ 実験
 - ・ 結果
 - ・ 考察
- ・ 最後に

アイデア：報酬空間を K 個の区間 (bin) に分割し、軌跡上で各 bin に属する報酬の個数を記録

手順：

1. 報酬の値を K 個の bin $B_1, B_2, \dots, B_K$ に離散化
2. 軌跡中の各報酬を bin にマッピング： $\text{count}_k = \text{bin } B_k \text{ に属した報酬の個数}$
3. bin 統計から下位 α パーセンタイルに相当する bin を特定
4. その bin 内の報酬の平均 $\Rightarrow$ CVaR の近似値

binning+CVaR の実装 (方法 1)

RRA の実装の枠組みでは基本的に、init/update/post の 3 つの関数の変更によって報酬をデザインできる。実際、以下のようにすることで binning による CVaR の近似を実装可能。

- init : 前半は各ビンの出現回数 (counts)、後半は各ビンの報酬合計 (sums) とする $2 \times K$ 次元のベクトルを 0 で初期化
- update : 新しい報酬値で counts,sums を更新
- post : sums/counts で、各ビンの平均を計算、counts から下位 α パーセンタイルに相当するビン群を特定、そのビン群の平均を CVaR の近似値として返す

学習時には binning によって計算される CVaR の近似値を用い、評価時とテスト時には真の CVaR を計算して性能を評価する。

binning+CVaR の実装 (方法 2)

- init : 各ビンの出現回数 (counts) とする K 次元のベクトルを 0 で初期化
- update : 新しい報酬値で counts を更新
- post : counts から下位 α パーセンタイルに相当するビン群を特定、それぞれのビン群には**ビン中心**を代表値として割り当て、その加重平均を CVaR の近似値とする

statistics の次元を半減できる。しかし、bin 内の実際の報酬についての情報は使わないので、精度は落ちる。

ニューラルネットワークなどによって真の関数を学習する際に生じる誤差

RRA の文脈での発生箇所：

価値関数 $v_{\pi}(s) = \text{post}(\tau_{\pi}(s))$ を Actor-Critic で学習する際、critic は $\tau_{\pi}(s) \in T$ (次元 d_{τ}) を近似する。

statistics の次元 d_{τ} が大きいとき、同じ訓練データから多次元の出力を学習するため、各次元あたりの推定精度が落ちる

目次

- ・ イントロ
- ・ 題材とした研究
- ・ やったこと
 - ・ 概要
 - ・ 手法
 - ・ 実験
 - ・ 結果
 - ・ 考察
- ・ 最後に

ベースラインとしては、

- Sharpe Ratio を最適化するように学習したモデル

を用いる。これは単に実装が正確にできているかの確認で、評価、テスト部分を真の CVaR に変えるだけで実装した。

bin 数変更による平均性能の推移

目的：CVaR の近似精度と function approximation error のトレードオフを実験的に評価する

実験設定：

- 手法：binning+CVaR（方法 1）
- bin 数の変動： $K \in \{25, 51, 101, 201, 401, 801\}$
- 評価、テスト時には真の CVaR で性能を評価
- 最終的な評価指標：テストにおける全期間での平均 CVaR 性能

期待される結果：

1. K が増加すると、初期段階では近似精度が向上 $\Rightarrow$ 性能向上
2. しかし、statistics の次元が増加（ $2K$ 次元）するにつれて **function approximation error** が増加
3. ちょうどいい bin 数が存在

binning+CVaR 実装の変更による性能の違い

2 つの実装方法の比較：

	方法 1	方法 2
statistics の次元	$2K$	K
ビン内情報	sums と counts を保持	counts のみ
近似精度	高い	低い
function approx. error	大きい	小さい

期待される結果：

- 方法 1 の利点：ビン内の情報を活用 $\Rightarrow$ より正確な CVaR 近似
- 方法 2 の利点：統計量次元を半減 $\Rightarrow$ 価値関数学習が容易に
- トレードオフ：どちらの効果が勝つかは実験的に検証が必要

実験セットアップ

データセット: RRA と同様

ウィンドウ: 訓練 5 年、評価 1 年（ベストモデル選択用）、テスト 1 年を 1 年ずつスライド（計 10 ステップ）

環境: 初期資本\$100,000、観測＝過去 59 日のログリターン（ 59×12 ）＋現在のアロケーション＋`vol_20`＋`vol_20_60_quotient`＋過去 57 日の VIX、報酬＝ステップごとの単純リターン $R_t = (P_{t+1} - P_t) / P_t$

CVaR: $\alpha = 0.05$

PPO: RRA と同じ

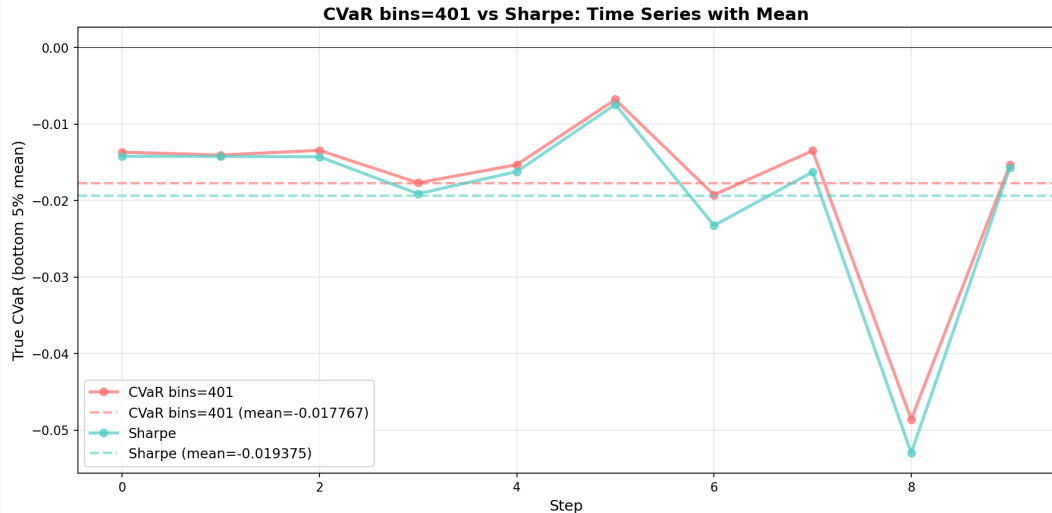
シード: 4 固定

計算環境: RTX3060Ti VRAM 8GB、1 回の学習に約 7,8 時間

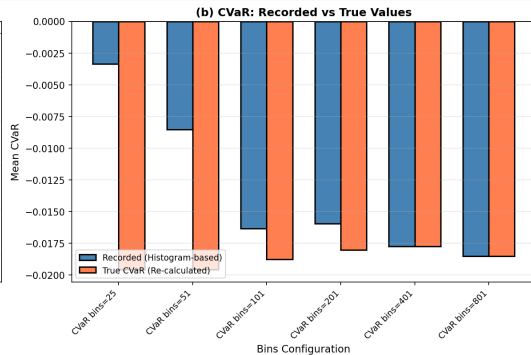
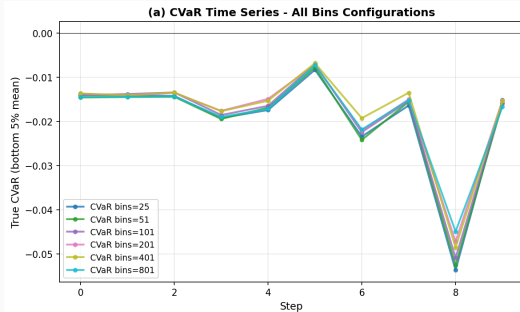
目次

- ・ イントロ
- ・ 題材とした研究
- ・ やったこと
 - ・ 概要
 - ・ 手法
 - ・ 実験
 - ・ 結果
 - ・ 考察
- ・ 最後に

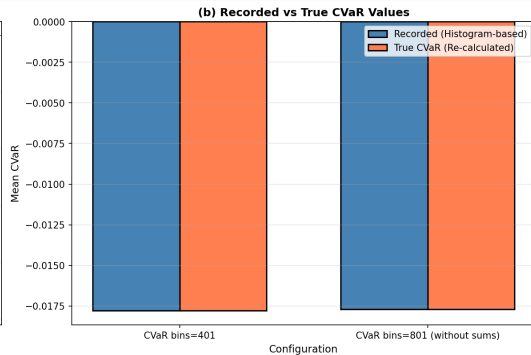
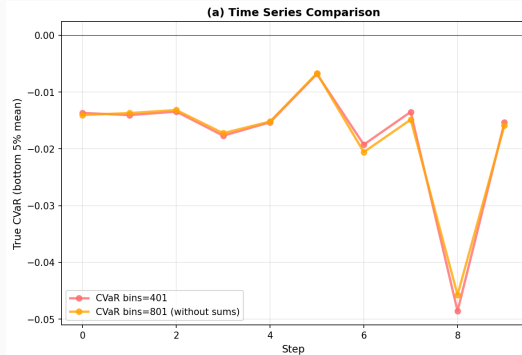
ベースラインとの比較



bin 数変更による平均性能の推移と近似誤差



binning+CVaR 実装の変更による性能の違い (未完成)



目次

- ・ イントロ
- ・ 題材とした研究
- ・ やったこと
 - ・ 概要
 - ・ 手法
 - ・ 実験
 - ・ 結果
 - ・ 考察
- ・ 最後に

1. RRA 枠組みでの CVaR 最適化の有効性

- Sharpe 最適化モデルと比較して、近似 CVaR モデルは真の CVaR で全期間・全バックテストで優れた性能を実現
- 結論：近似であっても RRA の枠組みで CVaR のような指標を効果的に最適化可能

2. CVaR 近似精度と性能のトレードオフ

- bin 数を増やすと近似精度が向上 $\Rightarrow$ 性能向上 ($K = 401$ まで)
- $K = 401$ で近似がほぼ真の CVaR に収束
- $K = 801$ での性能悪化：**function approximation error** が支配的に

3. 実装方法の比較（未完成）

- 801 の方法 2 と 401 の方法 1（同次元）で比較 $\Rightarrow$ 平均性能に有意差なし

注記：これらはシード 1 固定での結果。RRA のように、複数シード平均による統計的検証が必要。

目次

- ・ イントロ
- ・ 題材とした研究
- ・ やったこと
- ・ 最後に

やりたかったこと

以下のことをやろうと考えていたが、時間が足りなかった。

- RRA の実験のように複数シードの平均として結果を出したかった
- 実装の違いによる性能の違いを完成させたかった
- 下位 α パーセンタイルの bin についてのみ adaptive に bin を細かくすることで CVaR の近似精度を向上させられるか？
- REINFORCE や GRPO といった、pure policy gradient(つまり、モンテカルロ法なので、真の CVaR を直接最適化できるが、分散の悪化が期待される) の手法を実装して、近似なしでの CVaR 最適化を行い性能を比較する
- binning 以外の近似手法を使って、手法を比較する (例えば、分位点に相当する報酬を更新していく方法)
- 既存の CVaR 近似手法を調べて、その実装をベースラインとする
- crypto や crisis period の含まれるデータセットを用いることで、downside risk が重要な場面における CVaR 最適化の有用性をより明確にする

強化学習の概要、特に方策勾配法の手法や、その金融応用の取り組みについて知ることができた。さらに、RRA という報酬設計の枠組みを知り、それに CVaR を近似的に扱う方法を実装することで、CVaR を最適化できることが分かったので、特にダウンサイドリスクが重要な状況でのより整合的な目的関数を最適化できることが分かった。また、近似精度と function approximation error のトレードオフを実験的に確認することができた。

杉山先生には演習 3 全体の方向性について CVaR という指標を導入するモチベーションや指標の提案という方向性など、たくさんのアドバイスをいただいた。石田先生には特に最初の部分において、強化学習の基礎の習得に有用な英語の文献や、日本語の文献を教えていただいた。そもそも、この Recursive Reward Aggregation の論文は、Yivan Zhang さんに提案していただき、方向性についてたくさんのアドバイスをいただき、binning による CVaR 近似という方向性、それによって生じると期待される function approximation error と近似精度のトレードオフについて教えていただいた。また、ポートフォリオ部分の実装に当たって、Yuting Tang さんのコードをベースにさせていただき、実装や、実験のセットアップ方法についてたくさんのアドバイスをいただいた。

-  OpenAI (2026). ***Part 2: Kinds of RL Algorithms.*** URL:
<https://spinningup.openai.com/en/latest/> (visited on 05/14/2026).
-  Tang, Yuting (2024). ***Recursive Reward Aggregation.***
<https://github.com/Tang-Yuting/recursive-reward-aggregation>. URL:
<https://github.com/Tang-Yuting/recursive-reward-aggregation>.
-  Tang, Yuting et al. (2025). ***Recursive Reward Aggregation.*** arXiv: 2507.08537
[cs.LG]. URL: <https://arxiv.org/abs/2507.08537>.
-  康毅, 斎藤 (2022). ***ゼロから作る Deep Learning 4 強化学習編.*** オライリー・ジャパン. ISBN: 978-4-87311-975-5. URL:
<https://www.oreilly.co.jp/books/9784873119755/>.